# Homework 4 (CSCI-B 504)

Kushal Pokharel

November 2025

## 1 Problem 1

This problem is to let you get familiar with GPG (Gnu Privacy Guard), a free replacement for PGP (Pretty Good Privacy) so that you can send and receive emails to your friends/colleagues securely in the future. You need to:

1. set up your GPG for your "outlook" email system or others if you can.
2. extract your own public key information.
3. send a regular email to our helper Kushal at kupokh@iu.edu by Friday 11/1/24.
   The subject is "xxx's public key"
   where xxx is your name. Attached is your public key file.
4. then you will receive an email from Kushal, both encrypted and signed within a few days.
5. Kushal will send his public key to cs504-indy-l@list.iu.edu and you need to import his public key into your public key ring.
6. after receiving the above email (both encrypted and signed), you need to reply the received email to him within the NEXT day, both encrypting and signing your replying email.
   - exactly original email content from Kushal.
   - briefly describe the principle of GPG.

   NOTE: A good way to do this assignment is to find one partner in the class to test the steps first.

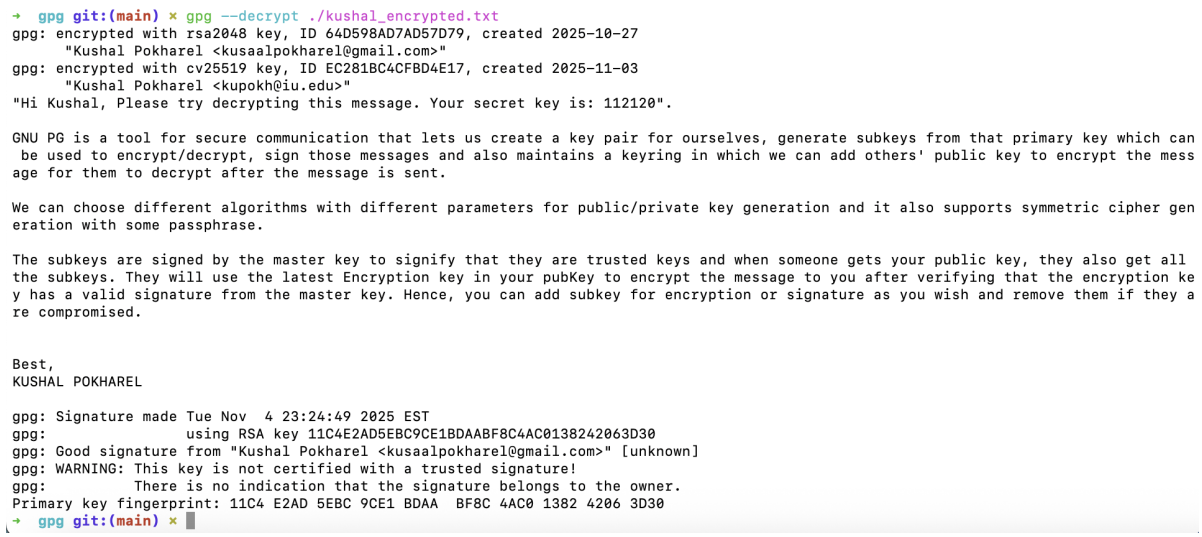


Figure 1: Decrypt with GPG

## 2 Problem 2

Please create your own personal webpage/homepage on server in-info- web4.luddy.indianapolis.iu.edu. The account for you on the server has been created and you can log in to it via your university cridential using ssh (such as ssh yourUniversityID@in-info-web4.luddy.indianapolis.iu.edu). Notes: if you login from outside the campus, you need to run the IU VPN first (if not, you need to download and install it first. Here is information on IU VPN: VPN ). Then create a directory and set its access user name and password to protect the files under the directory. Please submit the URL of your personal website and the user name and password to access the directory. The URL of your personal webpage will be https://in-info-web4.luddy.indianapolis.iu.edu/ yourID/

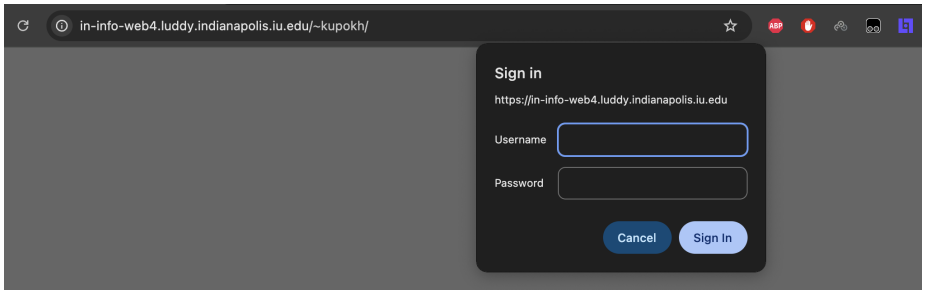


Figure 2: Login page with .htaccess

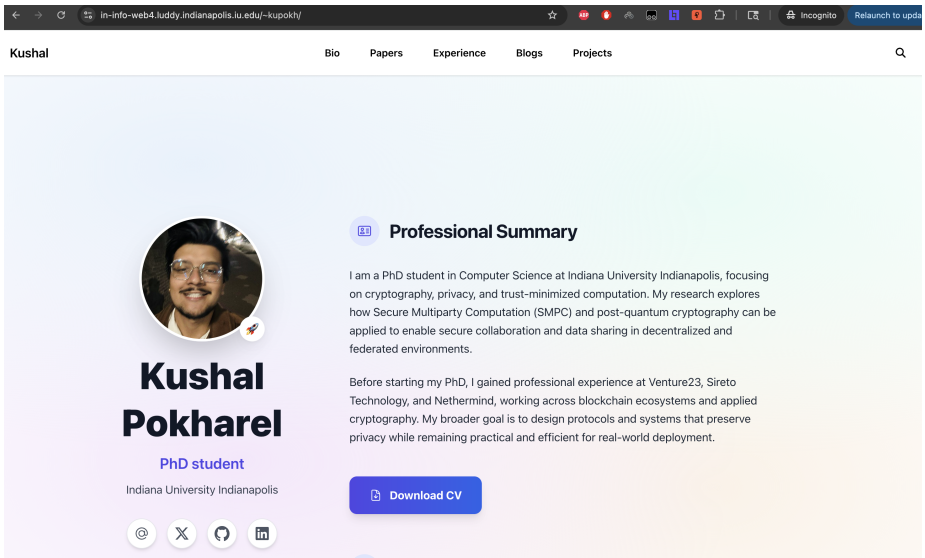


Figure 3: Webpage served by Apache webserver and protected by htaccess

- URL:

  `https://in-info-web4.luddy.indianapolis.iu.edu/~kupokh/`

- Username: kushal
- Password: Random

# 3 Problem 3

Any browser stores some public key certificates. You can view and edit them, such as delete, import, and export a certificate. Please view and manipulate them to get deep understanding about certificates and https. In particular, please find a particular website and when you access it using https, your browser shows that the website can not be verified and ask you for whether you want to risk to go to the website or to cancel the access. The reason behind the failed verification is that the certificate of the website is not stored already in the browser or is issued by a certificate authority (maybe by itself) whose public key (or to say, public key certificate) does not exist in your browser. (Notes: the public key of a root-level authority is stored as a self-signed certificate).

1. Please list the root-level certificates and the second-level certificates stored in your browser. And furthermore, draw the certificate hierarchy of multiple levels if possible.

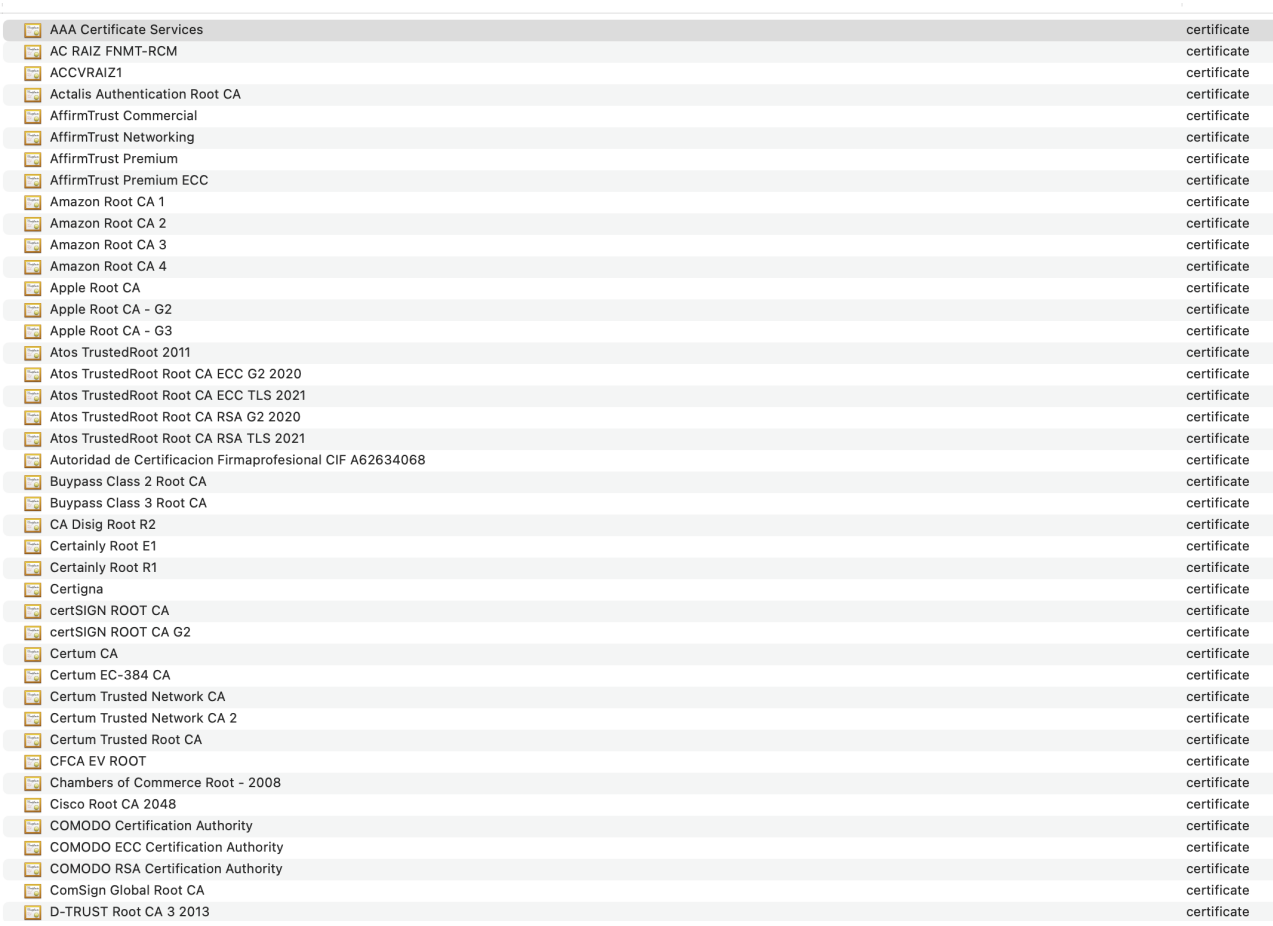


Figure 4: Root level Certificate Authority

These are the root-level certificate authorities stored in my PC. A subset of the most frequently occurring certificates is displayed in the browser; specifically, six were shown.

I also found a few intermediate certificate authorities stored in my system. One of the examples is:

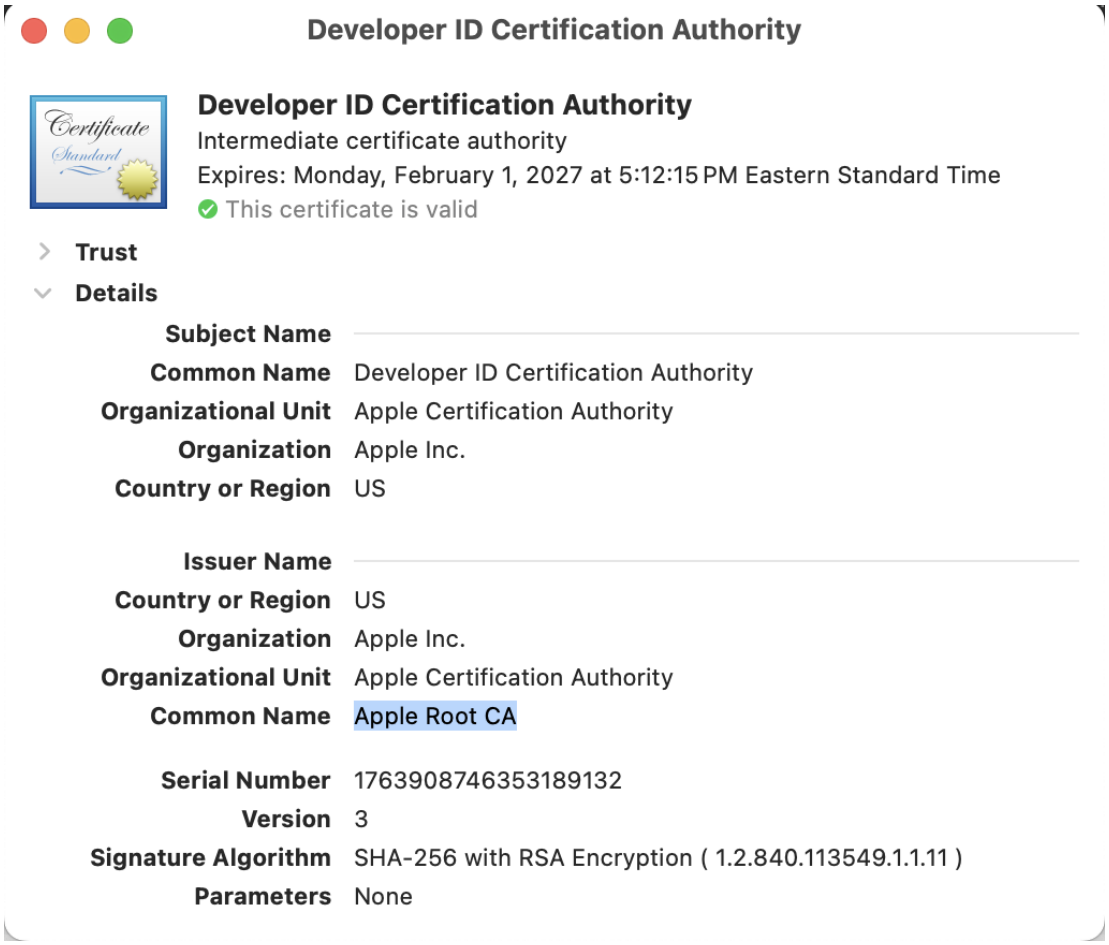


Figure 5: Intermediate CA example

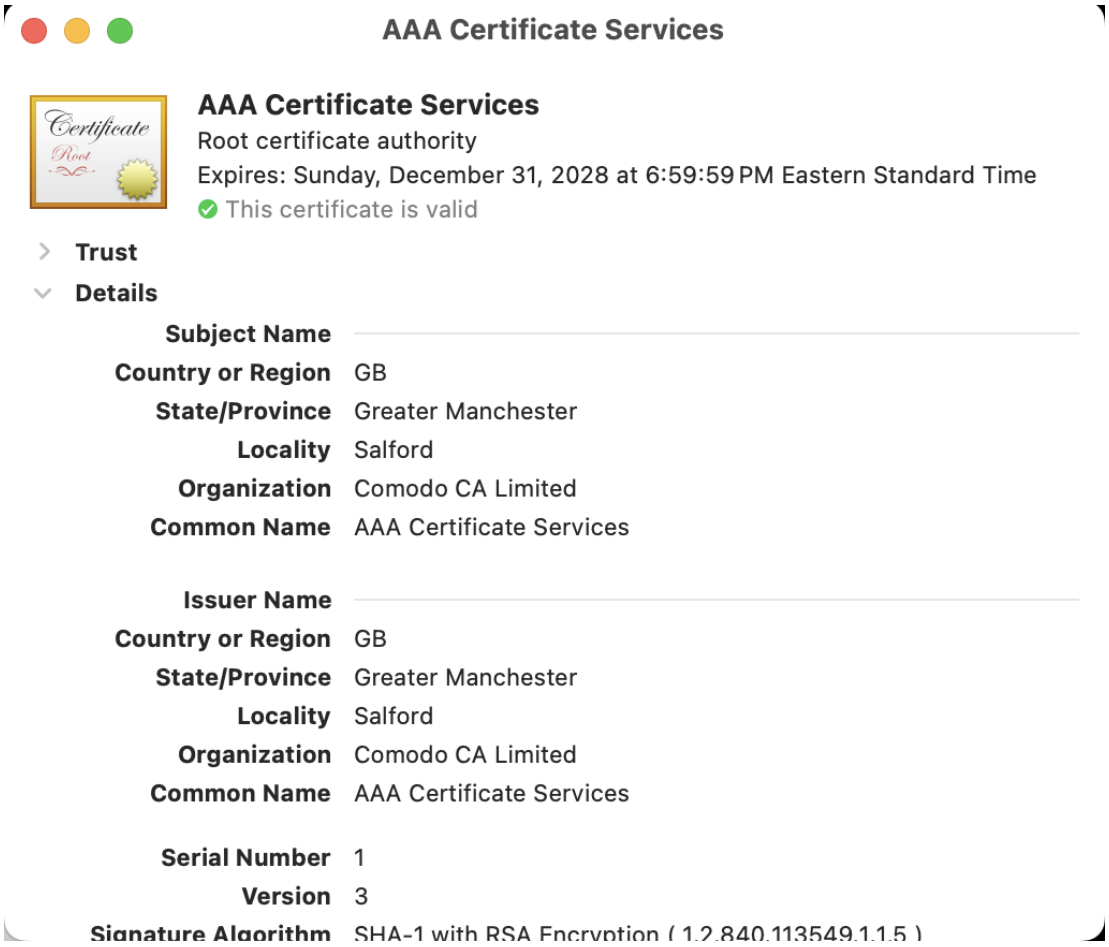


Figure 6: Root CA example

Comparing the intermediate and root CA, we find that the root CA is the ultimate source of trust maintained by trustworthy organizations and vetted by the OS, such as iOS and Windows. And intermediate CAs are frequently appearing certificate issuers that are trusted by the root CA. We can see that Developer ID CA was issued a certificate by the Apple Root CA.

```
[→  kushalpokharel.github.io git:(main) × openssl s_client -showcerts -servername kushalpokharel99.com.np -connect kushalpokharel99.com.np:443
Connecting to 2606:4700:3035::6815:21eb
CONNECTED(00000005)
depth=2 C=US, O=Google Trust Services LLC, CN=GTS Root R4
verify return:1
depth=1 C=US, O=Google Trust Services, CN=WE1
verify return:1
depth=0 CN=kushalpokharel99.com.np
verify return:1
---
Certificate chain
 0 s:CN=kushalpokharel99.com.np
   i:C=US, O=Google Trust Services, CN=WE1
   a:PKEY: EC, (prime256v1); sigalg: ecdsa-with-SHA256
   v:NotBefore: Sep 20 07:59:08 2025 GMT; NotAfter: Dec 19 08:56:44 2025 GMT
-----BEGIN CERTIFICATE-----
MIIDxTCCA2ygAwIBAgIQUAcvAu/iq10Oip50F6LIzDAKBggqhkjOPQQDAjA7MQsw
CQYDVQQGEwJVUzEeMBwGA1UEChMVR29vZ2xlIFRydXN0IFNlcnZpY2VzMQwwCgYD
VQQDEwNXRTEwHhcNMjUwOTIwMDc1OTA4WhcNMjUxMjE5MDg1NjQ0WjAiMSAwHgYD
VQQDExdrdXNoYWxwb2toYXJlbDk5LmNvbS5ucDBZMBMGByqGSM49AgEGCCqGSM49
AwEHA0IABJ0/Nl2ExpTsEEBQBv7Mvlw+KgpbaL5oDmlxYO2pz0mJbNKj/1OOHCHS
IbF5RGEVK6DkUQN75MOuaheINeGaD+mjggJpMIICZTAOBgNVHQ8BAf8EBAMCB4Aw
EwYDVR0lBAwwCgYIKwYBBQUHAwEwDAYDVR0TAQH/BAIwADAdBgNVHQ4EFgQUfbL7
9WQcBLfr3hUJynFC16JfpScwHwYDVR0jBBgwFoAUkHeSNWfE/6jMqeZ72YB5e8yT
+TgwXgYIKwYBBQUHAQEEUjBQMCcGCCsGAQUFBzABhhtodHRwOi8vby5wa2kuZ29v
Zy9zL3dlMS9VQWMwJQYIKwYBBQUHMAKGGWh0dHA6Ly9pLnBraS5nb29nL3dlMS5j
cnQwPQYDVR0RBDYwNIIXa3VzaGFscG9raGFyZWw5OS5jb20ubnCCGSoua3VzaGFs
cG9raGFyZWw5OS5jb20ubnAwEwYDVR0gBAwwCjAIBgZngQwBAgEwNgYDVR0fBC8w
LTAroCmgJ4YlaHR0cDovL2MucGtpLmdvb2cvd2UxL3Y3YkFlMHZXYVlzLmNybDCC
AQIGCisGAQQB1nkCBAIEgfMEgfAA7gB1AMz7D2qFcQll/pWbU87psnwi6YVcDZeN
tql+VMD+TA2wAAABmWZZPfwAAAQDAEYwRAIgAOo1rP3ViFSvydkw8GYwtMZv2YBr
6e8yjf2GVGAxn/ICIHFiZbM0CEOYb5Xw9RyP7B5mZBr98r1QAnd7hn2eW4PDAHUA
EvFONL1TckyEBhnDjz96E/jntWKHiJxtMAWE6+WGJjoAAAGZZlk9xQAABAMARjBE
AiEAtQxLdtLkCl5ZUH68W4GYgIVXuHbIs6xaKrsXu6TNc4gCH28kKQNU6Grdp8GA
ClvwVqPStpXhrjxfYOTR86ZgwEUwCgYIKoZIzj0EAwIDRwAwRAIhAO6bvLOpKXzn
9CiN8euNT6nABtP6wN5Vzr65SNKYL+onAh8fw69FUN3cTDiIiGhOS2Uxm/e70OrY
ryNoxw/OQifz
-----END CERTIFICATE-----
 1 s:C=US, O=Google Trust Services, CN=WE1
   i:C=US, O=Google Trust Services LLC, CN=GTS Root R4
   a:PKEY: EC, (prime256v1); sigalg: ecdsa-with-SHA384
   v:NotBefore: Dec 13 09:00:00 2023 GMT; NotAfter: Feb 20 14:00:00 2029 GMT
-----BEGIN CERTIFICATE-----
MIICnzCCAiWgAwIBAgIQf/MZd5csIkp2FV0TttaF4zAKBggqhkjOPQQDAzBHMQsw
CQYDVQQGEwJVUzEiMCAGA1UEChMZR29vZ2xlIFRydXN0IFNlcnZpY2VzIExMQzEU
MBIGA1UEAxMLR1RTIFJvb3QgUjQwHhcNMjMxMjEzMDkwMDAwWhcNMjkwMjIwMTQw
MDAwWjA7MQswCQYDVQQGEwJVUzEeMBwGA1UEChMVR29vZ2xlIFRydXN0IFNlcnZp
Y2VzMQwwCgYDVQQDEwNXRTEwWTATBgcqhkjOPQIBBggqhkjOPQMBBwNCAARvzTr+
Z1dHTCEDhUDCR127WEcPQMFcF4XGGTfn1XzthkubgdnXGhOlCgP4mMTG6J7/EFmP
LCaY9eYmJbsPAvpWo4H+MIH7MA4GA1UdDwEB/wQEAwIBhjAdBgNVHSUEFjAUBggr
BgEFBQcDAQYIKwYBBQUHAwIwEgYDVR0TAQH/BAgwBgEB/wIBADAdBgNVHQ4EFgQU
kHeSNWfE/6jMqeZ72YB5e8yT+TgwHwYDVR0jBBgwFoAUgEzW63T/STaj1dj8tT7F
avCUHYwwNAYIKwYBBQUHAQEEKDAmMCQGCCsGAQUFBzAChhhodHRwOi8vaS5wa2ku
Z29vZy9yNC5jcnQwKwYDVR0fBCQwIjAgoB6gHIYaaHR0cDovL2MucGtpLmdvb2cv
ci9yNC5jcmwwEwYDVR0gBAwwCjAIBgZngQwBAgEwCgYIKoZIzj0EAwMDaAAwZQIx
AOcCq1HW90OVznX+0RGU1cxAQXomvtgM8zItPZCuFQ8jSBJSjz5keROv9aYsAm5V
sQIwJonMaAFi54mrfhfoFNZEfuNMSQ6/bIBiNLiyoX46FohQvKeIoJ99cx7sUkFN
7uJW
-----END CERTIFICATE-----
 2 s:C=US, O=Google Trust Services LLC, CN=GTS Root R4
```

Figure 7: Certificate Chain for my website

We can see the summary of the certificate chain from the first part of the output, where we see at depth=0 my website has a certificate which is signed by the WE1 service presented at depth=1, and finally at depth=2, we have the root-level CA, GTS Root R4.
This root-level CA is one of the

2. What is the URL of the HTTPS website you visited? Describe what happen when you access this https website and how do you deal with it.

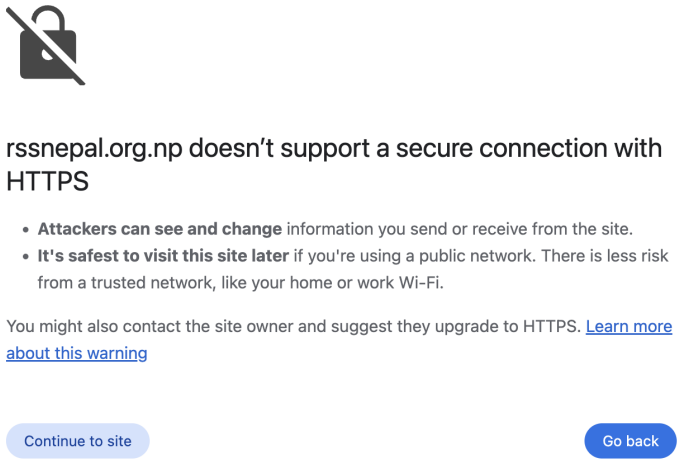


Figure 8: Insecure connection without TLS

This is the website of one of the popular news agencies in Nepal. Basically, we get this sort of warning when the browser can't verify the certificate chain presented by the website. This could mean that the end user (I) could be facing a man-in-the-middle attack that's presenting a fake certificate to me, and if I insert any sensitive information, the middleman can decrypt and gain access to my sensitive information.

Usually, when I find something like this, I enter the website if it's important, but I don't put in any sensitive information; I just surf around. It automatically becomes a read-only website for me.

# 4 Problem 4.1

**A signature in the DSA is not allowed to have $\gamma = 0$. Show that if a "signature" in which $\gamma = 0$ is known, then the value of k used in that "signature" can be determined. Given the value of k, show that it is now possible to forge a "signature" (with $\gamma = 0$) for any desired message (i.e., a selective forgery can be carried out).**

Overview of DSA:

- Let p be a L-bit prime (L=0 mod 64 and $512 \leq L \leq 1024$) such that DLP in $Z_p^*$ is intractable and q be a 160-bit prime dividing p-1. Let $\alpha \in Z_p^*$ be the $q^{th}$ root of 1 modulo p.
  This defines a generator in the subgroup mod q where $\alpha$ is the generator of the subgroup. Isomorphism between the $(Z_q,+)$ and the multiplicative subgroup G where $G = \{\alpha^k \pmod p \mid k \in \{0, 1, \ldots, q-1\}\}$
- Let Message space=0,1*, and signature space = $Z_q^* \times Z_q^*$, define
  - Key space=$(p, q, \alpha, a, \beta : \beta = \alpha^a (mod p)$ and $a \in [0, q-1]$
  - $p, q, \alpha, \beta$ are public key and a is private key.
- Let $h : \{0,1\}^* \in Zq$ be a secure hash function
- For $K = (p, q, \alpha, a, \beta)$ and for a random k, $k \in \{1, q-1\}$ define:
  - $Sig_K(x, k) = (\alpha, \beta)$ where (if $\alpha$=0 or $\gamma$=0,a new k is chosen):
  - $\gamma$=($\alpha^k$ mod p mod q) and $\delta = (SHA1(x) + a\gamma)k^{-1}$ mod q.
- For $x \in \{0,1\}^*$ and $\gamma, \delta \in Z_q^*$, define
  – $ver_K(x, (\gamma, \delta))$ =true iff $\alpha^{e1}\beta^{e2}$ mod p mod q = $\gamma$ where
  - e1=$SHA1(x)\delta^{-1}$ mod q and $e2 = \gamma\delta^{-1}$ mod q.

To show, when $\gamma = 0$, k can be determined and it is possible to forge a signature for any message.

If $\gamma = 0$, we have $\delta = SHA1(x)k^{-1}$ mod q, then $\delta k = SHA1(x)$.
Since we know message x and also the algorithm SHA1 is public, we can easily find the hash of the message and finally find $k$ by dividing the hash by $\delta$ in mod q. Finally, we can then
we can pair (0, $SHA1(x)k^{-1}$) and send for any message x. This will always result in correct verification of the signature.

## 4.1 Problem 4.2

Here is a variation of the ElGamal Signature Scheme. The key is constructed in a similar manner as before: Alice chooses $\alpha \in Z_p^*$ to be a primitive element, chooses a secret integer $a \in Z_{p-1}$ where $\gcd(a, p-1) = 1$, and computes $\beta = \alpha^a \pmod p$. The key $K = (\alpha, \beta, p)$ where $\alpha$ and $\beta$ are the public key and $a$ is the private key.

Let $x \in Z_p$ be a message to be signed. Alice computes the signature $\text{sig}(x) = (\gamma, \delta)$, where

$$\gamma = \alpha^k \pmod{p} \quad \text{and} \quad \delta = (x - k\gamma)a^{-1} \pmod{p-1}.$$

The only difference from the original ElGamal Signature Scheme is in the computation of $\delta$. Answer the following questions concerning this modified scheme:

1. **Describe how a signature $(\gamma, \delta)$ on a message $x$ would be verified using Alice's public key.**

   Using Alice's public key $\beta$, the signature can be verified with $\alpha^x = \beta^\delta \gamma^\gamma$, all of which are publicly available to us.
   Correctness:
   From delta, $\delta = (x - k\gamma)a^{-1} => \delta a = (x - k\gamma) => x = \delta a + k\gamma$
   Hence,
   $\alpha^x = \alpha^{\delta a + k\gamma} = \alpha^{a\delta}\alpha^{k\gamma} = \beta^\delta \gamma^\gamma \pmod{p}$

2. **Describe a computational advantage of the modified scheme over the original scheme.**
   The difference from the original scheme is based on $\delta$, in which calculation of $a^{-1}$ in this modified protocol only needs to be done once since we have the constant private key a rather than having to calculate $k^{-1}$ in the original protocol which keeps changing in each signature genration. Hence this protocol has that computational advantage of not having to calculate the inverse over mod $p - 1$ for each signature generation.

   While in signature verification, the computational complexity is almost the same.

3. **Briefly compare the security of the original and modified scheme.**

   In the modified scheme, in $\delta$, one of the factor is always same as described above. Hence when we divide two $\delta$s from the two signature of two messages, the constant factor gets cancelled and we can get an inside information on the difference of $k$ values used, which will ultimately allow us to do some operations on k and figure out the private key - a. And the system is broken. The hacker can generate as many signatures impersonating the user.

   $\frac{\delta_1}{\delta_2} = \frac{(x_1 - k_1\gamma_1)a^{-1}}{(x_2 - k_2\gamma_2)a^{-1}} = \frac{(x_1 - k_1\gamma_1)}{(x_2 - k_2\gamma_2)}$

   $(x_2 - k_2\gamma_2)\delta_1 = (x_1 - k_1\gamma_1)\delta_2 \Rightarrow \ (x_2 - k_2\gamma_2)\delta_1 = (x_1 - k_1\gamma_1)\delta_2$

   All of the above values except $k_2$ and $k_1$ is known so we get some equation on $k_2$ and $k_1$ like:
   $ak_1 + bk_2 = c$ where a, b and c are constants.
   But we can't solve this yet, to produce system of equations we take another signature on message $x_3$ and do the same calculation as above with those messages and signatures with both $x_1$ and $x_2$, which will give us three equations for three unknown variables which can easily be solved. After

we get one $k$, the $\delta$ trivially gives us the value of a. And the system is broken.

# 5 Problem 5

We discussed different Internet security protocols from application layer to network layer. However, we did not discuss data link layer security protocol yet. For data link layer, the focus is on wireless/wi-fi transmission since radio transmission is easy to intercept/eavesdrop. Please search, learn and discuss wireless security and corresponding protocol(s) for data link layer.

We have discussed various Internet security protocols ranging from the application layer (TLS) down to the network layer (IPsec). However, the security of **Data Link Layer (Layer 2)** is critical because, as mentioned, the **Wifi/wireless waves** can easily be intercepted by some attackers in proximity, as the radio waves are easy to intercept. The security protocols at this layer are designed to provide **local, hop-to-hop protection** between a device and the wireless Access Point (AP). The process is generally to encrypt the data from the IP layer and add a Layer-2 header specifying the next hop (subsequent routers). The next hop then decrypts the IP packet, determines the next hop, and finally encrypts the packet for that hop, which continues until the packet reaches its destination.

## Wireless Security Protocols for Data Link Layer

The primary security protocols for the IEEE 802.11 (Wi-Fi) standard fall under the Wi-Fi Protected Access (WPA) family, which evolved to address critical cryptographic weaknesses.

### Evolution of Standards

The evolution showcases the shift from broken ciphers to robust, AES-based protection:

| Protocol | Status | Core Cipher | Security Summary |
|---|---|---|---|
| **WEP** | Obsolete/Broken | RC4 Stream Cipher | Initial standard, easily cracked due to static keys and weak Initialization Vectors (IVs). |
| **WPA** | Obsolete | TKIP / RC4 | An interim patch; improved key management but relied on the weak RC4 stream cipher. |
| **WPA2** | Widespread Standard | CCMP / AES | Introduced the robust AES cipher. Remains highly secure, though its key handshake has known implementation vulnerabilities (e.g., KRACK). |
| **WPA3** | Current Standard | GCMP / AES | Mandatory for new devices. Addresses WPA2 flaws and introduces superior key exchange. |

## Focus on WPA3: The Current Standard

WPA3 is the most current and secure protocol for Data Link Layer protection, specifically mitigating the primary attacks against prior standards:

### 1. Stronger Key Exchange (SAE)

WPA3 replaces the vulnerable Pre-Shared Key (PSK) handshake of WPA2 with the Simultaneous Authentication of Equals (SAE) protocol (Dragonfly).

- **Result:** SAE makes the network resistant to offline dictionary attacks. Even if an attacker captures the initial authentication traffic, they cannot efficiently brute-force the password offline, thereby mitigating the risk of weak user-chosen passwords.

### 2. Enhanced Confidentiality for Open Networks

WPA3 introduced Opportunistic Wireless Encryption (OWE) for public or guest networks that do not use a password.

- **Result:** OWE automatically encrypts the traffic between the user and the AP. The Diffie-Hellman Key Exchange is performed initially between the devices to get a shared key even without a password. While it does not provide authentication, it guarantees **confidentiality** against passive eavesdropping, preventing attackers from easily reading session traffic on open Wi-Fi hotspots.

**3. Mandatory Management Frame Protection (PMF)**

WPA3 requires the use of Protected Management Frames (PMF).

- **Result:** This cryptographic signing of administrative frames prevents Layer 2 Denial-of-Service attacks, specifically deauthentication and disassociation attacks, which rely on sending spoofed, unauthenticated packets to disconnect users.